The Vanguard Arm

Theory, Decisions, and Lessons from a Mars Rover Manipulator

Project Vanguard — BITS Pilani Hyderabad

Krithin Poola   Claude (documentation & reference)

Started July 2026 — a living document

Contents

Part I

The Build Journal

Chapter 1

Why This Stack

This book documents the design, theory, and hard lessons of the Vanguard rover arm — manipulation autonomy for the BITS Pilani Hyderabad Mars Rover team, built to win IRC 2027 and grow into a research platform. Every chapter is written in the same session the work happened, so the reasoning here is what we actually thought, not a cleaned-up retrospective.

1.1 The mission, worked backwards

The arm exists to score points in four competitions (IRC, ARC, ERC, URC) whose manipulation tasks — studied from the primary rulebooks — collapse into six reusable skills: grasp & carry, panel operations, connector insertion, oriented placement, sampling support, and read-&-report. IRC's IDMO panel (buttons, switches, knobs, a 3-pin plug), ARC's propellant hose, ERC's RJ-45, and URC's board connectors are one skill family wearing four costumes. The central architectural bet follows: *build skills, not missions.* A new rulebook becomes a re-orchestration, not a rebuild.

Two strategy decisions bound everything else:

1. **Reliable assisted teleoperation wins IRC 2027; genuine autonomy is the ERC 2027 goal.** Top teams win on reliability and operator skill. Autonomy is invested exactly where the rulebook prices it (IRC's RADO delivery: autonomous = 100% of points, teleoperated = 50%).

2. **Simulation first.** The physical arm arrives months after work begins. The stack is therefore built against an Isaac Sim digital twin exposing

identical ROS 2 interfaces to the future hardware — so hardware arrival is an integration event, not a starting gun. This same twin later becomes the benchmark environment of our first paper.

1.2 One mental model

A manipulator is a chain of rigid-body transforms in $SE(3)$, and every tool in the stack is that one mathematical object viewed differently. The URDF *declares* the chain. TF2 *broadcasts* it. Forward kinematics *composes* it:

$$T_{\mathrm{ee}}(\theta) \;=\; e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} \cdots e^{[\mathcal{S}_n]\theta_n} M,$$

the product of exponentials over the joint screw axes $\mathcal{S}_i$ acting on the home pose M. Inverse kinematics *inverts* it — nonlinear, multi-solution, and ill-conditioned near singularities. The Jacobian *differentiates* it, $\mathcal{V} = J(\theta)\dot{\theta}$, and is why teleop jogging, visual servoing, and singularity handling are the same problem. MoveIt 2 *plans* through the chain; `ros2_control` *executes* along it; Isaac Sim *simulates* it from the same URDF. Ten tools, one idea.

Two consequences we will meet repeatedly:

- **Six degrees of freedom is the floor, not a luxury.** Connector insertion requires commanding an arbitrary approach *orientation* at an arbitrary position — six constraints. A 5-DoF arm can reach the socket but cannot, in general, present the plug square to it.

- **Singularity is a property of the map, not the motor.** At a straight elbow, J loses rank; some Cartesian velocities become unreachable and naive IK commands explode. We damp (damped least squares) rather than pretend precision the hardware doesn't have.

1.3 Infrastructure decisions (recorded before the first line of code)

Platform. Ubuntu 22.04 + ROS 2 Humble for the IRC season; migration to Jazzy/24.04 is *scheduled* (post-IRC window, Humble EOL May 2027) rather than improvised.

Containers. Development happens in a devcontainer (`ros:humble-ros-base-jammy`); the rover will run a lean L4T-based runtime image on the Jetson. Deployment is `git pull && docker compose up`, not dependency archaeology at 7 am in a field.

DDS. CycloneDDS as the team RMW, `--network=host --ipc=host` on every container, one `ROS_DOMAIN_ID`. This dissolves the classic "container can't discover the robot" failure class before anyone hits it.

TensorRT engines are built on the device that runs them. Engines are not portable artifacts; on-rover engine generation is a scripted first-boot step (a lesson inherited directly from the APEX project).

Bring-up is native; runtime is containerized. When a motor doesn't enumerate you want `dmesg` with zero layers in between. Containers earn their keep once drivers are stable.

Compute. Development and Isaac scenes on a 12 GB RTX laptop; heavy training (GR00T-class fine-tunes, RL) as containerized batch jobs on the university HPC. The nightly simulation regression runs on the docked laptop.

1.4 Mistakes log, opened early

This section exists in every chapter. Failures are recorded with root causes because they are the highest-density teaching material we produce.

Lesson. The paste-boundary error (day one). *The very first typed file of the project — `.devcontainer/Dockerfile.dev` — ended up containing the JSON of `devcontainer.json` pasted below the Dockerfile instructions; the build would have died on the first JSON line. Found in review before first build. Root cause: transcribing two reference files in one editing pass without a file-boundary check. Rule adopted: after typing from reference,* verify each file builds/parses on its own *before moving to the next. The same failure family (paste-indent) cost a debugging session on APEX in June 2026.*

Chapter 2

The Stand-In Arm

2.1 Why a UR5e we don't own

Phase 0 needs an arm months before mechanical delivers one. The first plan was a hand-written placeholder xacro; Krithin counter-proposed adopting an existing, battle-tested description — and the counter-proposal won the review. The Universal Robots UR5e matches our envelope almost exactly (850 mm reach, 5 kg payload ≈ the IRC cache spec), ships an official MoveIt configuration and an official NVIDIA Isaac Sim asset, is the arm every ROS 2 tutorial assumes, and is the reference target of the GELLO teleoperation design we intend to build. There is also a research reason: our first paper's benchmark is more reproducible if its baseline arm is one any lab can load.

The stand-in comes with guardrails so UR conveniences cannot hide problems our real arm will have:

- **Generic IK only** (KDL / TracIK / pick_ik) — never the UR-specific analytic solver. Mech's arm will not have the UR's near-ideal wrist geometry.
- **Skills address `tool0` and named planning groups**, never joint names. Swapping in the real URDF must be a configuration change, not a refactor.
- **Gates re-run on swap.** Every skill success-rate measured on the UR5e is re-measured on the real description the week it lands.

2.2 The two-process architecture (what confused us, drawn once)

The first launch attempt produced no GUI and a confused operator. The reason is the most important architectural fact in ROS 2 manipulation:

Terminal 1: the driver `ur_control.launch.py`		**Terminal 2: the planner** `ur_moveit.launch.py`
`ros2_control_node`	$\xleftarrow{\text{FollowJointTrajectory action}}$	MoveIt `move_group`
hardware interface (mock)	$\xrightarrow{\texttt{/joint\_states}}$	RViz + MotionPlanning
controllers (JTC, broadcasters)		

The driver *owns the hardware* (here 'mock_components', later our CAN drivers on the Jetson) and exposes controllers; MoveIt is merely a *client* of the `FollowJointTrajectory` action. They are separate processes speaking DDS — which is why they can run in one container, two containers, or a laptop and a rover 500 m apart, unchanged. On competition day this same diagram is the base station (planner/UI) and the rover (driver); the fake-hardware flag is the only line that changes.

Working vocabulary pinned by this exercise: `controller_manager` loads and activates controllers against the hardware interface; `joint_state_broadcaster` publishes what the encoders (here, simulated) report; `scaled_joint_trajectory_controller` executes time-parameterized joint trajectories. The "No GUI?" moment resolved itself the instant the second process started — the GUI was never missing, it simply hadn't been asked for.

2.3 No gripper — by design

The stock `ur_description` ends at `tool0`, a bare flange: industrial arms ship without end-effectors because the end-effector is application-specific. This mirrors our own architecture decision (interface spec §5): the gripper is a *swappable module* on a standard flange, developed on its own track by mechanical. In simulation we will attach a Robotiq 2F-85 description at `tool0` when grasping work begins — it is the standard research gripper, its ROS 2 description is public, and its parallel-jaw morphology matches what mech is likely to build first. Until then, MoveIt plans to `tool0` poses, which is exactly how the skills should address the arm anyway.

2.4 Mistakes log

Lesson. Prescribing a launch file that doesn't exist (Claude's error). *The instruction said* `demo.launch.py`*; Humble's* `ur_moveit_config` *ships* `ur_moveit.launch.py` *and expects a separately-launched driver. Root cause: quoting a newer distro's package layout from memory instead of checking the installed package. Rule: before prescribing a launch, verify it —* `ls $(ros2 pkg prefix <pkg>)/share/<pkg>/launch/`*. This check belongs in the planned* `ros2-debug` *skill.*

Lesson. Ephemeral installs in `--rm` ***containers.*** `ur_robot_driver` *was installed live inside a running container started with* `--rm`*; it will vanish with the container. Anything installed interactively must be promoted to the Dockerfile the same day — the image is the single source of truth for the environment.*

Lesson. Placeholders are for replacing. `docker exec -it <that-name> bash` *was typed verbatim, angle brackets and all. Trivial, universal, worth stating once:* `< >` *in any command means "substitute your value." Every teammate onboarding guide gets this sentence.*

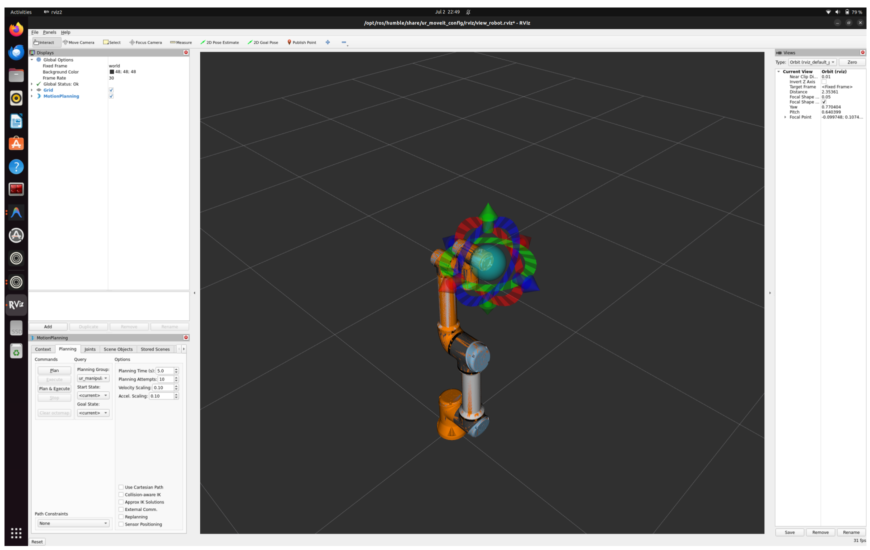

Figure 2.1: First motion plan, 2 July 2026: UR5e on mock hardware inside the devcontainer. MoveIt's MotionPlanning panel with the `ur_manipulator` group; the interactive marker (rings and sphere) sets the `tool0` goal pose. Velocity and acceleration scaling deliberately at 0.10 — competition speed comes later, correctness comes first.

Chapter 3

Speaking to Controllers

3.1 The action interface every skill rides

Chapter 2 established that MoveIt is just a client of the driver. This chapter removes MoveIt from the loop: a 50-line `rclpy` node (`ros2_ws/src/scripts/send_trajectory.py`) built a `trajectory_msgs/JointTrajectory` by hand and sent it to `/scaled_joint_trajectory_controlle` as a `control_msgs/action/FollowJointTrajectory` goal. The arm moved. That one exchange is the load-bearing interface of the entire stack: every future skill — grasp, panel-op, insertion — ultimately terminates in exactly this action call.

Anatomy of the message, as used:

- `joint_names` — an *ordered* list; position i in every waypoint binds to name i. The controller maps names to its configured joints, so the order in the message is yours to choose — but the positions must match *your* chosen order exactly.
- `points[]` — waypoints, each with `positions` and `time_from_start`. The controller spline-interpolates between them (Humble's JTC: 'splines' method); you specify *where* and *when*, it decides the in-between.
- The action (not topic!) gives a goal/feedback/result contract: the client learns whether execution succeeded (`error_code=0`) — the primitive on which skill-level retry logic will be built.

Timing is the velocity command: the same two waypoints with `time_from_start` of 8 s vs. 5 s trace the same path at different speeds — velocity is emergent (Δposition/Δtime), not commanded. Competition pacing will be tuned exactly here.

3.2 Mistakes log

Lesson. Loud bug: wrong module. *`from trajectory_msgs.action import ...` — `JointTrajectory` lives in `trajectory_msgs.msg`. Instant `ImportError`; cost: seconds. Interface types live in `.msg`, action definitions in `control_msgs.action`; when unsure, `ros2 interface show <type>`.*

Lesson. Silent bug: a set where a list belongs. *`UR_JOINTS` was typed with braces `{}` — a Python* set, *which iterates in hash order. Since waypoint positions bind to joint names* by index, *a scrambled name order silently assigns the shoulder's −1.57 to a wrist. In simulation, a curiosity; on hardware beside a panel, a collision. Two rules adopted: (1) the future skills layer gets a trajectory sanity-checker (names, limits, monotonic timing) in front of every action goal; (2) fail-loud beats fail-scrambled — prefer interfaces that reject wrong types over ones that reinterpret them.*

3.3 Experiment results (observed 2 July 2026, mock hardware)

Timing. With `p2` moved from t=8 s to t=5 s, the first leg (start→`p1`, 4 s) was unchanged and the second leg ran in 1 s instead of 4 — visibly 4× faster over the identical path. Velocity was never commanded; it emerged from waypoint spacing. (Observed exactly as predicted by the interpolation model above.)

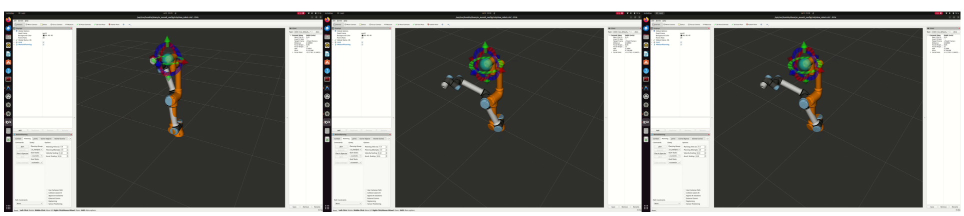

Figure 3.1: Frames from the screen recording of the two-waypoint trajectory executing in RViz — the arm sweeping from home-ish through the elbow-bent midpoint to the target pose, commanded by `send_trajectory.py` with no MoveIt in the loop.

The limit experiment: an impossible motion, executed. Commanding the elbow to 4.0 rad — ∼49° past its ±π URDF limit — produced neither a rejection nor a clamp: *the mock arm simply performed the impossible motion.* Three layers each declined to protect us, and knowing which is which matters:

1. **The trajectory controller** interpolates and forwards; Humble's JTC does not validate goals against URDF position limits.

2. **Mock hardware** accepts any command — that is what "mock" means.

3. **MoveIt**, the layer that *does* respect URDF limits when planning, was deliberately not in the loop — our raw action client bypassed the only guard present.

On the real UR, the vendor controller would have refused. On *our* arm, there is no vendor controller — we are the vendor. Consequences adopted:

- The Phase-1 hardware interface enforces joint limits (position, velocity, effort) in `read/write`, independent of anything upstream.

- The skills layer's trajectory sanity-checker (chapter's first lesson) gains a limits check — names, limits, monotonic timing — in front of *every* action goal.

- Simulation passing is not evidence of safety when the simulation layer doesn't model the constraint. Know what each sim tier actually enforces (mock < Isaac physics < hardware).

Chapter 4

The Digital Twin Breathes

4.1 What was built

Isaac Sim 5.1 now simulates the UR5e under PhysX, and ROS 2 commands it: an OmniGraph action graph publishes the simulated encoders at 60 Hz (`/joint_states`, measured 59.99 Hz, σ=0.9 ms) and feeds subscribed `JointState` commands into an Articulation Controller. A hand-published message from a terminal moved the simulated arm. The digital twin of chapter 1 stopped being a diagram on 5 July 2026.

The graph is five nodes: *On Playback Tick* drives *ROS2 Publish Joint State*, *ROS2 Subscribe JointState* → *Articulation Controller*, with *Read Simulation Time* stamping headers. The stage lives in the repo (`ros2_ws/isaac/VanguardArm.usd`); a bonus discovery in its tree: the 5.1 UR5e asset ships *with a gripper* (`Gripper` prim + `robot_gripper_joint`), so grasping work gets simulated fingers for free.

4.2 The debugging ledger (four traps, all now labeled)

Lesson.* `omniverse:///` *is a server you don't have. *Save As defaults to a Nucleus URL; a bare filename typed there fails with* "Cannot save layer... Error: Connection" *— and took the whole action graph with it the first night (unsaved stage, Isaac closed). Rule: in any Omniverse file dialog, click* My Computer *and verify the path bar shows a local path* before *naming the file. Corollary: save the stage before building anything on it.*

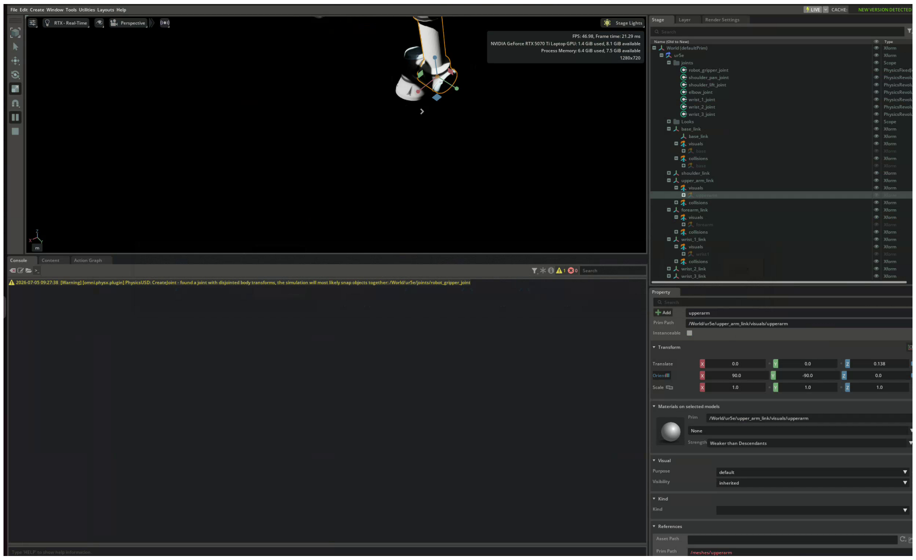

Figure 4.1: First ROS-commanded motion in Isaac Sim: the UR5e mid-sweep after a `JointState` message published from the host terminal. Right: the asset's true structure in the Stage tree — links, joints scope, and the `root_joint` that turned out to carry the Articulation Root.

Lesson. USD prim paths are case-sensitive. *Graph nodes targeting `/World/ur5e` while the prim differed only in case produced hundreds of* Failed to find articulation *errors — and an empty ROS topic list that masqueraded as a DDS problem. Debugging order adopted: read the prim path from the Stage panel and set targets with the eyedropper, never by typing.*

Lesson. errno 28 lies. *Thousands of* "No space left on device" *errors at startup with 255 GB free: inotify* watch-limit *exhaustion (`fs.inotify.max_user_watches`, default 65,536 — Isaac watches every extension directory). Fixed permanently via sysctl (1M watches). The error string reports ENOSPC; the device that is "full" is a kernel table, not the disk.*

Lesson. The Articulation Root is not where you point. *With the path case fixed, physics still found no articulation: on fixed-base Isaac assets the `ArticulationRootAPI` sits on the* fixed base joint *(`/World/ur5e/root_joint`), not the robot's outer Xform. Both ROS nodes had to target the root joint. General rule: target the prim that* carries the API, *findable by clicking through the Stage tree and watching the Property panel for the "Articulation Root" section.*

4.3 Architecture note: why this layer stays thin

The action graph deliberately does *not* implement control: it is a transport shim — encoders out, position targets in. Control authority stays in `ros2_control`, which next connects to these topics via a `topic_based` hardware interface so that the *identical* controller stack (and `send_trajectory.py`, unmodified) runs against Isaac exactly as it ran against mock hardware. One contract, three bottoms: mock, Isaac, and — in September — motors. The topics are being renamed `/isaac_joint_states` and `/isaac_joint_commands` accordingly, because the plain `/joint_states` name belongs to `ros2_control`'s broadcaster in that stack.

Chapter 5

One Interface, Three Bottoms

5.1 The result first

On 5 July 2026, `send_trajectory.py` — unchanged since the night it was written against mock hardware — executed its two-waypoint trajectory on the Isaac-simulated UR5e and returned `error_code=0`, and the sweep was confirmed *visually* in the viewport. (The return code alone is weaker than it looks — see the tolerance lesson in this chapter's mistakes log; the eyes were the real verifier that day.) The chapter-1 promise — *sim and real expose identical interfaces* — is now a demonstrated property of the stack, not a design intention. Mock was the first bottom; Isaac is the second; September's motors are the third.

5.2 The anatomy of the overlay

Four small files (package `vanguard_isaac_control`) created the bridge. What each *is*, precisely:

`urdf/ur5e_isaac.urdf.xacro` — the noun. Xacro is URDF with macros; expansion produces one robot description with two conceptually distinct parts. The *kinematic* part (links, joints, limits) comes from UR's `ur_robot` macro reading their config YAMLs — chapter 7's PoE chain, serialized. The *<ros2_control>* part declares no geometry at all: per joint, which command/state *interfaces* exist, and which *hardware plugin* provides them. Ours is `topic_based_ros2_control/TopicBasedSystem`

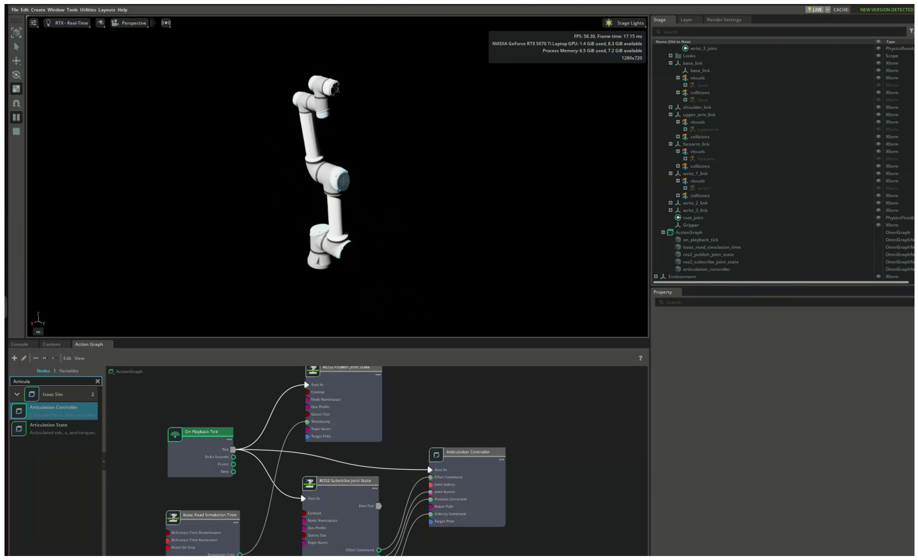

Figure 5.1: The full stack mid-trajectory: PhysX viewport, the OmniGraph bridge wiring below (tick → publish/subscribe → articulation controller), stage tree right.

with two parameters — the command and state topic names. That one block is the entire difference between the mock arm, the Isaac arm, and the future real arm.

`config/controllers.yaml` — the loop configuration. The controller manager (`ros2_control_node`) runs read→update→write at 60 Hz (matched to Isaac's tick). The YAML names its controllers: `joint_state_broadcaster` (republishes hardware state as `/joint_states` — which is why the Isaac-side topics were renamed `/isaac_*`; the plain name belongs to this broadcaster) and a `JointTrajectoryController` deliberately *instance-named* `scaled_joint_trajectory_controller` so the action path baked into `send_trajectory.py` resolves unchanged. Names are labels; types are behavior.

`launch/isaac_control.launch.py` — the verb. Starts four processes: `robot_state_publish` (live FK → TF), the controller manager (given the URDF's control block + the YAML), and two short-lived `spawners` that ask the manager — via services — to load, configure, and activate each controller, then exit. When the manager crashes, spawners wait for-ever on those services: that log signature (*"waiting for service /controller_manager/list_controllers"*) means *look upstream, the server died.*

The data path, named end to end: action goal → JTC spline interpolation → position commands through the hardware interface → `TopicBasedSystem` publishes `/isaac_joint_commands` → Isaac's Articulation Controller drives PhysX → `/isaac_joint_states` at 60 Hz → back through the plugin as controller feedback and out as `/joint_states` for everyone else. The script sees only the action; the bottom is invisible by construction.

5.3 Mistakes log

Lesson. A plugin is a promise someone must install. *First launch died with `pluginlib::LibraryLoadException`: the URDF requested `TopicBasedSystem`, but `ros-humble-topic-based-ros2-control` was never installed — the crash log helpfully listed every plugin that* did *exist, which is the fastest way to read this failure class. Same root pattern as the ephemeral-install lesson of chapter 2: the environment's source of truth is the Dockerfile, and it hadn't been rebuilt.*

Lesson. The description you include may bring its own controller. *The same crash log showed `Loading hardware 'ur5e'`* succeeding *before ours loaded: `ur_description` v2.10's macro generates its* own *`ros2_control` block by default (`generate_ros2_control_tag:=true`), so the robot had two hardware systems claiming the same six joints. Fix: one attribute, `generate_ros2_control_tag="false"`. Verification habit adopted: after any description change, `xacro ... | grep -c '<ros2_control'` must print exactly 1.*

Lesson. One letter, zero communication. *After everything reported success, the arm did not move: the controller published `/isaac_joint_commands` while Isaac subscribed to `/isaac_joint_command` — singular. Videos and stares found nothing; `ros2 topic info` found it in one command: `pub=1, sub=0` on one topic and the mirror image on its sibling is the exact signature of a name mismatch. Standing rule: when a topic chain is silent,* count the endpoints before watching the robot.

Lesson. "Success" is only as strong as its tolerances (a correction). *This chapter first claimed* `error_code=0` *proved PhysX tracked the trajectory within tolerance. Wrong — and the proof was an accident: a leftover experiment value (elbow* $= 4.0$ *rad, past its limit) also returned success. Humble's* `JointTrajectoryController` *checks goal tolerances only if the YAML configures* `constraints`*; ours didn't, so success meant "trajectory sent," not "arm arrived." PhysX clamped the impossible joint; nobody objected. Fix queued: explicit* `constraints` *per joint, after which* `error_code=0` *becomes an instrument reading instead of a pleasantry. Meta-lesson for the book itself: claims about what a check certifies must be verified against the check's configuration, not its name.*

Lesson. A trailing space after a backslash is invisible and fatal. *The Dockerfile's line continuation* `ros-humble-ur-robot-driver \␣` *(note the space* after *the backslash) breaks the multi-line* `RUN` *— the next line becomes a separate, invalid instruction. Found in review before it cost a build. Editors with show-whitespace would have flagged it; ours now does.*

5.4 Phase 0b: closed

Remaining Phase-0 work builds *on* this bridge rather than extending it: the IDMO panel scene in Isaac, MoveIt pointed at the overlay stack, the six skill stubs, teleop, and the LeRobot recording path for P1. The interface layer itself is done — and it is the most load-bearing thing this project will ever build, which is why it got five chapters.

Part II

Foundations — Block A Theory

Chapter 6

Rigid-Body Motion

Everything the arm does is described by rotations and translations of rigid bodies. This chapter is the vocabulary; chapters 7 and 8 are the grammar. Reference: Modern Robotics ch. 3 (Lynch & Park); read this chapter alongside it.

6.1 Rotations

A rotation matrix $R \in SO(3)$ is a 3×3 matrix satisfying

$$R^\top R = I, \qquad \det R = +1.$$

Its columns are the axes of the rotated frame expressed in the reference frame — that reading solves most sign confusions. Rotations compose by multiplication and *do not commute*: rotating 90° about $\hat{z}$ then $\hat{x}$ lands differently than $\hat{x}$ then $\hat{z}$. Order is meaning: $R_{ab}R_{bc} = R_{ac}$ (subscript chains cancel like units).

Representations we will actually touch.

- **Rotation matrix** — what TF2 and all math uses internally; 9 numbers, 3 DoF.
- **Axis–angle** $(\hat{\omega}, \theta)$ — Rodrigues' formula reconstructs R:

  $$R = I + \sin\theta\,[\hat{\omega}] + (1 - \cos\theta)\,[\hat{\omega}]^2, \tag{6.1}$$

  where $[\hat{\omega}]$ is the skew-symmetric matrix of $\hat{\omega}$ ($[\hat{\omega}]x = \hat{\omega} \times x$). This *is* the exponential map: $R = e^{[\hat{\omega}]\theta}$.

- **Unit quaternion** $q = \left(\cos\frac{\theta}{2},\ \hat{\omega}\sin\frac{\theta}{2}\right)$ — 4 numbers, no singularities, what ROS messages carry (`x,y,z,w` order in `geometry_msgs`, w last — a classic bug source). q and $-q$ are the same rotation (double cover).

- **Euler/RPY** — human-readable, gimbal-locked, used only at config-file boundaries (URDF `rpy=`). Never do math in RPY.

Example 6.1 (Quaternion of a 90° yaw)**.** $\hat{\omega} = (0, 0, 1)$, $\theta = \pi/2$: $q = (\cos 45°,\ 0, 0, \sin 45°) = (\frac{\sqrt{2}}{2}, 0, 0, \frac{\sqrt{2}}{2})$. In ROS message order: x=0, y=0, z=0.7071, w=0.7071.

6.2 Homogeneous transforms

Pose = rotation + translation, packed as $T \in SE(3)$:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}, \qquad T^{-1} = \begin{bmatrix} R^\top & -R^\top p \\ 0 & 1 \end{bmatrix}.$$

Composition chains frames: $T_{ab}\,T_{bc} = T_{ac}$. This subscript-cancellation rule is the whole of TF2: `lookup_transform(a, c)` multiplies the chain of stored transforms for you.

Key result 6.1. Never invert by transposing the whole T; only R transposes. The translation picks up $-R^\top p$.

Example 6.2 (Wrist camera to base)**.** The wrist camera sees an ArUco tag at $T_{\text{cam,tag}}$. The arm reports $T_{\text{base,tool0}}$; hand–eye calibration gave $T_{\text{tool0,cam}}$. Where is the tag for the planner?

$$T_{\text{base,tag}} \;=\; T_{\text{base,tool0}}\; T_{\text{tool0,cam}}\; T_{\text{cam,tag}}.$$

Every panel-servicing skill begins with exactly this triple product. A 2 mm error in the middle factor (calibration) shifts every tag estimate by 2 mm — which is why chapter B1 (hand–eye) exists.

6.3 Twists and screws (the 60-second version)

Angular and linear velocity pack into a twist $\mathcal{V} = (\omega, v) \in \mathbb{R}^6$. A unit screw axis $\mathcal{S} = (\hat{\omega}, v)$ with $v = -\hat{\omega} \times q$ (for a revolute joint through point q) generalizes "rotation about a line." Matrix exponentials of screws produce rigid displacements: $T = e^{[\mathcal{S}]\theta}$. That single fact powers the next chapter's forward kinematics. For deeper treatment: MR ch. 3.3; for the Lie-group view, Solà et al. (arXiv:1812.01537) — read it *after* this clicks numerically.

6.4 Problems

Problem 6.1. Show that $R_z(90°)R_x(90°) \neq R_x(90°)R_z(90°)$ by tracking where the unit vector $\hat{x}$ lands under each. (No matrices needed — rotate a pen.)

Answer. First ordering (apply R_x first, then R_z... careful: in a fixed frame, the *left* matrix acts last): $R_z R_x$ sends $\hat{x} \to R_x\hat{x} = \hat{x} \to R_z\hat{x} = \hat{y}$. The other ordering sends $\hat{x} \to \hat{y} \to$ (rotation about $\hat{x}$) $\to \hat{z}$. $\hat{y} \neq \hat{z}$: non-commutative.

Problem 6.2. Using (6.1), compute R for $\hat{\omega} = (0, 0, 1)$, $\theta = 90°$, and verify it maps $\hat{x} \to \hat{y}$.

Answer. $[\hat{\omega}] = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$, $\sin\theta = 1$, $1 - \cos\theta = 1$, $[\hat{\omega}]^2 = \mathrm{diag}(-1, -1, 0)$.
Sum: $R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$; first column $= \hat{y}$.

Problem 6.3. $T_{\text{base,tool0}}$ has $R = R_z(90°)$ and $p = (0.4, 0.1, 0.5)$. Compute T^{-1} and state what frame relationship it represents.

Answer. $R^\top = R_z(-90°)$; $-R^\top p = -R_z(-90°)(0.4, 0.1, 0.5) = -(0.1, -0.4, 0.5) = (-0.1, 0.4, -0.5)$. It is $T_{\text{tool0,base}}$: the base pose as seen from the tool.

Problem 6.4 (transfer)**.** Your teammate reports the gripper quaternion as $(0.7071, 0, 0, 0.7071)$ "in ROS order." What are the two possible rotations they might mean, and how do you disambiguate?

Answer. If (x, y, z, w): $x = 0.7071, w = 0.7071 \to 90°$ about $\hat{x}$. If they wrote (w, x, y, z): $90°$ about $\hat{z}$. Disambiguate by convention-checking against a known pose (or just `ros2 topic echo` and compare with RViz). Adopted team rule: always name the convention when writing quaternions down.

Chapter 7

Forward Kinematics: the Product of Exponentials

Forward kinematics answers: *given joint angles θ, where is the tool?* The URDF answers it numerically every time `robot_state_publisher` broadcasts TF; this chapter is what that computation actually is. Reference: MR ch. 4.

7.1 The formula

Choose a home configuration (all $\theta_i = 0$) and record $M \in SE(3)$, the tool frame at home. For each joint i, record its screw axis $\mathcal{S}_i$ (in the base frame, at home). Then

$$T_{\text{base,tool}}(\theta) \;=\; e^{[\mathcal{S}_1]\theta_1}\, e^{[\mathcal{S}_2]\theta_2} \cdots e^{[\mathcal{S}_n]\theta_n}\, M. \tag{7.1}$$

Each factor "turns on" one joint, in order from base to tip, then M places the tool. For a revolute joint through point q_i with axis $\hat{\omega}_i$: $\mathcal{S}_i = (\hat{\omega}_i,\ -\hat{\omega}_i \times q_i)$.

Why PoE and not Denavit–Hartenberg: no frame-assignment ritual, no four-parameter bookkeeping, axes read straight off the CAD/URDF, and it composes cleanly with everything in chapter 6. (DH still appears in older papers and UR's own documentation — recognize it, don't adopt it.)

7.2 Worked example: the 2R planar arm

Two links, lengths l_1, l_2, both joints about $\hat{z}$, arm along $\hat{x}$ at home.

Setup. $M = \begin{bmatrix} I & (l_1{+}l_2, 0, 0)^\top \\ 0 & 1 \end{bmatrix}$; $\mathcal{S}_1 = (0, 0, 1;\ 0, 0, 0)$ (axis through origin); joint 2 passes through $q_2 = (l_1, 0, 0)$: $v_2 = -\hat{z} \times q_2 = -(0, l_1, 0)$, so $\mathcal{S}_2 = (0, 0, 1;\ 0, -l_1, 0)$.

Result (expanding (7.1), or by trigonometry):

$$x = l_1 \cos\theta_1 + l_2 \cos(\theta_1{+}\theta_2), \qquad y = l_1 \sin\theta_1 + l_2 \sin(\theta_1{+}\theta_2). \tag{7.2}$$

Sanity limits (always do these): $\theta = (0, 0)$ gives $(l_1{+}l_2,\ 0)$ — home. $\theta = (0, 180°)$ gives $(l_1 - l_2,\ 0)$ — folded back. $\theta = (90°, 0)$ gives $(0,\ l_1{+}l_2)$ — straight up.

Example 7.1 (Numbers)**.** $l_1 = l_2 = 1$, $\theta = (30°, 45°)$: $x = \cos 30° + \cos 75° = 0.8660 + 0.2588 = 1.1248$; $y = \sin 30° + \sin 75° = 0.5 + 0.9659 = 1.4659$.

7.3 The UR5e through this lens

The UR5e is six revolute joints; its screw axes come from the frame data in `ur_description` (the `_kinematics.yaml` + xacro chain you met in chapter 2). Structurally: joint 1 yaws about the base $\hat{z}$; joints 2, 3, 4 pitch about parallel $\hat{y}$-axes (shoulder, elbow, wrist-1 — this parallel-axis triple is why the arm has an elegant "elbow-up/elbow-down" geometry); joint 5 rolls; joint 6 pitches the flange. When mech's arm arrives, our first analytical act will be writing ITS six screw axes from the CAD — the PoE table *is* the kinematic datasheet.

Key result 7.1. The URDF is a serialized PoE model: each `<joint>`'s `<origin>` + `<axis>` encode where q_i and $\hat{\omega}_i$ sit relative to the parent. FK in `robot_state_publisher`, planning in MoveIt, and physics in Isaac all consume this one description — which is why URDF correctness (chapter 2's guardrails, mech's CAD export) is load-bearing for everything.

7.4 Problems

Problem 7.1. For the 2R arm with $l_1 = 0.35$, $l_2 = 0.30$ (our interface-spec proportions), compute the tool position at $\theta = (45°, -30°)$.

Answer. $\theta_1{+}\theta_2 = 15°$. $x = 0.35 \cos 45° + 0.30 \cos 15° = 0.2475 + 0.2898 = 0.5373$; $y = 0.35 \sin 45° + 0.30 \sin 15° = 0.2475 + 0.0776 = 0.3251$.

Problem 7.2. Add a base yaw joint (about $\hat{z}$ at the origin, but now the 2R plane is vertical: joints 2, 3 about $\hat{y}$). Write the three screw axes $\mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3$ at home (arm along $\hat{x}$).

Answer. $\mathcal{S}_1 = (0,0,1;\, 0,0,0)$; $\mathcal{S}_2 = (0,1,0;\, -\hat{y} \times q_2)$ with $q_2 = 0 \Rightarrow v_2 = 0$ if the shoulder sits at the origin, else $q_2 = (0,0,h)$ gives $v_2 = -\hat{y} \times (0,0,h) = (-h,0,0)$; $\mathcal{S}_3 = (0,1,0;\, -\hat{y} \times (l_1,0,h)) = (0,1,0;\; -h,0,l_1)$. (If you got $v_3 = (-h,0,l_1)$ check the cross-product order: $\hat{y} \times (l_1,0,h) = (h,\, 0,\, -l_1)$, negated $\Rightarrow (-h,0,l_1)$.)

Problem 7.3. Why must the exponentials in (7.1) be multiplied in base-to-tip order? What physical statement does swapping $e^{[\mathcal{S}_1]\theta_1}$ and $e^{[\mathcal{S}_2]\theta_2}$ make?

Answer. Each screw is written in the *home* base frame; turning joint 1 moves joint 2's axis through space, and left-multiplication applies that displacement to everything downstream. Swapping claims joint 2 rotates about its home axis regardless of joint 1 — a different (wrong) machine. Same reason matrix order mattered in Problem 1.1.

Problem 7.4 (bridge to practice)**.** In RViz you set the UR5e joints to all-zero with `joint_state_publisher_gui`. The arm points straight up, not along $\hat{x}$. What does that tell you about M in `ur_description`, and why does it not matter for anything downstream?

Answer. UR's home configuration has the arm vertical — M encodes that choice. PoE is indifferent: M is just "where the tool is at zero." Conventions only need to be consistent between the description and everything that consumes it — which the URDF guarantees by construction.

Chapter 8

Jacobians, Singularities, and Inverse Kinematics

Chapter 7 maps joint angles to tool pose. This chapter differentiates that map — and inverts it. Teleop jogging, MoveIt's Cartesian goals, visual servoing (Block C), and the straight-elbow experiment from chapter 3 all live here. Reference: MR ch. 5–6.

8.1 The Jacobian

Differentiating FK gives a linear map from joint velocities to tool twist:

$$\mathcal{V} = J(\theta)\,\dot{\theta}, \qquad J \in \mathbb{R}^{6\times n}. \tag{8.1}$$

Column i answers: *if only joint i moves at unit speed, how does the tool move, at this configuration?* For a revolute joint currently on axis $\hat{\omega}_i$ through point q_i, with tool at p:

$$J_i = \begin{bmatrix} \hat{\omega}_i \\ \hat{\omega}_i \times (p - q_i) \end{bmatrix}.$$

The linear part is lever-arm physics: velocity = axis × arm. Long arm, fast tip — the same reason the shoulder joint dominates our reach envelope and the interface spec's payload-at-extension clause exists.

8.1.1 Worked example: the 2R Jacobian

Differentiate (7.2) directly:

$$J(\theta) = \begin{bmatrix} -l_1 s_1 - l_2 s_{12} & -l_2 s_{12} \\ l_1 c_1 + l_2 c_{12} & l_2 c_{12} \end{bmatrix}, \qquad \det J = l_1 l_2 \sin\theta_2. \tag{8.2}$$

($s_{12} = \sin(\theta_1 + \theta_2)$, etc.)

8.2 Singularities

$\det J = l_1 l_2 \sin\theta_2$ vanishes at $\theta_2 = 0$ or π: the **straight (or folded) elbow**. There the two columns become parallel — both joints can only move the tip *perpendicular* to the arm; no joint motion produces velocity *along* the arm. Consequences, in increasing order of practical pain:

1. A whole Cartesian direction is instantaneously unreachable.
2. Near (not at) the singularity, reaching the "hard" direction requires enormous $\dot\theta$: $\dot\theta = J^{-1}v$ blows up as $\det J \to 0$.
3. Numerical IK oscillates or explodes if implemented naively.

The **manipulability** $w(\theta) = \sqrt{\det(JJ^\top)}$ ($= |l_1 l_2 \sin\theta_2|$ for the 2R) measures distance from trouble; MoveIt Servo watches a version of this and scales velocity down as w shrinks — that is the guard rail you will feel when jogging the UR5e through a straight-arm pose.

Key result 8.1. Singularity is a property of the *configuration*, not the target. The fix is never "push harder": it is damping (below), path shaping (approach panels with a bent elbow — operator playbook material), or redundancy (a 7th joint, which we don't have).

8.3 Inverse kinematics

IK inverts FK: given desired T_d, find θ. For our arms (and any arm without special structure — see chapter 2's generic-IK guardrail) this is numerical:

Newton–Raphson: iterate $\theta \leftarrow \theta + J^{+}(\theta)\, e(\theta)$ where e is the pose error (twist from current to desired) and J^{+} the pseudoinverse. Fast far from singularities; violent near them.

Damped least squares (DLS / Levenberg–Marquardt):

$$\Delta\theta = J^\top \left(JJ^\top + \lambda^2 I\right)^{-1} e. \tag{8.3}$$

The damping λ trades tracking accuracy for bounded joint velocity: as $JJ^\top$ loses rank, $\lambda^2 I$ keeps the inverse finite, so near-singular configurations produce slow, sane motion instead of a joint-speed spike. This is the "we damp rather than pretend precision" line from chapter 1, now with its formula. TracIK / KDL / pick_ik are production-grade wrappers around exactly this idea plus restarts and limits handling.

8.3.1 Worked example: one NR step, by hand

2R arm, $l_1 = l_2 = 1$, at $\theta = (0°, 90°)$, tool at $(1, 1)$ by (7.2). Target: (1.2, 1.0), so $e = (0.2,\ 0)$.

From (8.2) with $s_1{=}0, c_1{=}1, s_{12}{=}1, c_{12}{=}0$:

$$J = \begin{bmatrix} -1 & -1 \\ 1 & 0 \end{bmatrix}, \quad \det J = 1, \quad J^{-1} = \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix}, \quad \Delta\theta = J^{-1}e = (0,\ -0.2)\ \text{rad}.$$

New $\theta = (0°,\ 78.54°)$; FK gives (1.199, 0.980). The x-error is nearly closed in one step; y picked up a small error because the step is first-order — iteration exists precisely to mop that up. Two or three more steps converge to sub-millimeter.

8.4 Problems

Problem 8.1. For the 2R arm at $\theta = (30°, 0°)$ (straight arm), compute J and exhibit a Cartesian velocity no $\dot\theta$ can produce.

Answer. $s_{12} = s_1 = \frac{1}{2}$, $c_{12} = c_1 = \frac{\sqrt{3}}{2}$: $J = \begin{bmatrix} -\frac{1}{2} - \frac{1}{2} & -\frac{1}{2} \\ \frac{\sqrt{3}}{2} + \frac{\sqrt{3}}{2} & \frac{\sqrt{3}}{2} \end{bmatrix} = \begin{bmatrix} -1 & -\frac{1}{2} \\ \sqrt{3} & \frac{\sqrt{3}}{2} \end{bmatrix}$; columns are parallel (second = first $\times\frac{1}{2}$), $\det J = 0$. Both columns point perpendicular to the arm direction $(\cos 30°, \sin 30°)$; any velocity *along* the arm, e.g. $v = (\cos 30°, \sin 30°)$, is unreachable.

Problem 8.2. In the NR worked example, redo the step with DLS (8.3) and $\lambda = 0.5$. Compare $\|\Delta\theta\|$ with the undamped step and explain the difference in one sentence.

Answer. $JJ^\top = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$; $JJ^\top{+}0.25I = \begin{bmatrix} 2.25 & -1 \\ -1 & 1.25 \end{bmatrix}$, inverse $= \frac{1}{1.8125}\begin{bmatrix} 1.25 & 1 \\ 1 & 2.25 \end{bmatrix}$. Then $(JJ^\top{+}\lambda^2 I)^{-1}e = \frac{1}{1.8125}(0.25,\ 0.2) = (0.1379, 0.1103)$ and $\Delta\theta = J^\top(\cdot) = (-0.1379{+}0.1103,\ \ -0.1379) = (-0.0276, -0.1379)$. $\|\Delta\theta\| = 0.141$ vs. 0.2 undamped: damping shrinks (and slightly rotates) the step, buying bounded velocity at the cost of extra iterations.

Problem 8.3. The UR5e refused to jog smoothly through a fully-extended pose in MoveIt Servo (you will try this). Which mechanism from this chapter is acting, and what are the operator's two escapes?

Answer. Servo's singularity scaling: manipulability collapsed at full extension, so it throttles Cartesian velocity toward zero. Escapes: re-approach along a direction the arm *can* move (perpendicular), or jog in joint space through the singular region and resume Cartesian control on the far side. Playbook rule: plan panel approaches with a bent elbow.

Problem 8.4 (looking ahead to Block C)**.** Visual servoing will command tool velocities from image error: $v = -\kappa\, L^{+} e_{img}$. Trace the full chain from a pixel error to motor current using the objects defined in chapters 6–8 and the interfaces from chapter 3.

Answer. Pixel error $\to$ interaction matrix L^{+} $\to$ desired camera twist $\to$ (adjoint/frame transform, ch. 6) tool twist $\mathcal{V}$ $\to$ DLS inverse of J (ch. 8) $\to$ $\dot{\theta}$ $\to$ velocity/position commands to the controller (ch. 3) $\to$ driver $\to$ current. Every box is now named; Block C fills in L.